

머신러닝의 핵심: 결정 경계부터 최적화까지, 수학적 원리 완벽 가이드

목차

서론: 머신러닝, 왜 수학으로 시작해야 할까?

1. 머신러닝의 목표: 데이터 공간을 가르는 '결정 경계'

특성 공간(Feature Space)의 개념

결정 경계(Decision Boundary)의 정의

결정 경계의 종류

2. 최고의 경계를 찾는 여정: 손실 함수와 최적화

손실 함수(Loss Function)의 필요성

대표적인 손실 함수: 힙지 손실(Hinge Loss)

최적화(Optimization)와 경사 하강법(Gradient Descent)

3. [핵심] 결정 경계를 만드는 알고리즘: 판별 모델 파헤치기

판별 모델(Discriminative Model)이란?

가장 단순한 시작: 퍼셉트론(Perceptron)

최적의 마진을 찾아서: 서포트 벡터 머신(SVM)

확률적 접근: 로지스틱 회귀(Logistic Regression)

4. 모델 비교: 판별 모델 vs. 생성 모델

두 모델의 근본적인 차이점

언제 무엇을 사용해야 할까?

5. 실전! 코드로 결정 경계 시각화하기 (날씨 예측 예제)

목표 및 프로세스

결정 경계 시각화 코드 및 해설

결론 및 핵심 용어 정리

핵심 요약 및 향후 학습 방향

핵심 용어 정리 (Glossary)

서론: 머신러닝, 왜 수학으로 시작해야 할까?

인공지능은 어떻게 고양이와 개를 구별할까요? 이 질문은 머신러닝의 핵심 목표 중 하나인 '분류 (Classification)' 문제로 이어집니다. 기계가 수많은 데이터 속에서 패턴을 발견하고 특정 기준에 따라 대상을 구분하는 원리는, 단순히 복잡한 코드를 넘어 명확하고 논리적인 수학적 원리에 깊이 뿌리내리고 있습니다. 머신러닝 모델의 학습 과정은 본질적으로 데이터(점)들을 가장 잘 나눌 수 있는 '최적의 결정선(Decision Boundary)'을 찾는 수학적 탐험과 같습니다.

이 글에서는 머신러닝의 근간을 이루는 이 '결정 경계'라는 개념을 중심으로, 경계선을 어떻게 수학적으로 정의하고(모델 가설), 그 경계선이 얼마나 좋은지 평가하며(손실 함수), 마침내 최적의 선을 찾아가는지(최적화)에 대한 전 과정을 시각적인 자료와 함께 체계적으로 탐험하고자 합니다. 이 여정을 통해 머신러닝의 작동 원리를 보다 근본적으로 이해하게 될 것입니다.

학습 목표:

- 머신러닝의 근본적인 목표인 '결정 경계'의 개념을 이해합니다.
- 모델의 성능을 측정하는 '손실 함수'의 역할을 파악합니다.
- 최적의 모델을 찾아가는 '경사 하강법'의 원리를 직관적으로 이해합니다.
- 판별 모델(Discriminative Model)의 대표적인 예시인 퍼셉트론, SVM 등을 통해 결정 경계가 어떻게 만들어지는지 학습합니다.

고양이와 개의 이미지를 '색상/질감', '모양/크기' 등의 특성으로 변환하여 2차원 공간에 표현하고, 두 그룹을 나누는 결정 경계를 시각화한 예시

1. 머신러닝의 목표: 데이터 공간을 가르는 '결정 경계'

머신러닝, 특히 지도 학습(Supervised Learning)의 분류 문제에서 핵심 목표는 주어진 데이터를 명확한 기준에 따라 구분하는 것입니다. 이를 이해하기 위해서는 먼저 데이터가 어떻게 수학적으로 표현되고, 그 공간에서 '경계'가 어떤 의미를 갖는지 알아야 합니다.

특성 공간(Feature Space)의 개념

이미지, 텍스트, 소리와 같은 비정형 데이터는 기계가 직접 이해할 수 없습니다. 따라서 우리는 '특성 추출(Feature Extraction)'이라는 과정을 통해 데이터를 기계가 처리할 수 있는 숫자 벡터(vector) 형태로 변환합니다. 이렇게 변환된 데이터 벡터들이 존재하는 다차원 공간을 ****특성 공간(Feature Space)****이라고 부릅니다. **특성 공간은 모델이 이해할 수 있는 모든 가능한 특성 벡터들의 집합입니다.**

예를 들어, 사람을 '키'와 '몸무게'라는 두 가지 특성으로 표현한다면, 각 사람은 2차원 특성 공간(평면) 위의 한 점으로 나타낼 수 있습니다. 마찬가지로 고양이 사진 한 장도 '털의 평균 색상값', '귀의 뾰족한 정도', '눈 사이의 거리' 등 수백, 수천 개의 특성으로 변환될 수 있으며, 이는 곧 수백, 수천 차원의 고차원 특성 공간 속의 한 점으로 표현됩니다. **이러한 방식으로 기계는 구체적인 개념(예: 고양이)을 학습하게 됩니다.**

결정 경계(Decision Boundary)의 정의

결정 경계(Decision Boundary)란 특성 공간에서 서로 다른 클래스(예: 개 vs. 고양이, 스팸 메일 vs. 정상 메일)에 속하는 데이터 포인트들을 구분하는 경계면을 의미합니다. **이 경계를 시각화하는 것은 모델이 데이터를 어떻게 '바라보고' 분류 결정을 내리는지 직관적으로 이해하는 데 매우 효과적인 방법입니다.**

머신러닝에서 '모델(Model)' 또는 '가설(Hypothesis)'은 바로 이 결정 경계를 정의하는 특정 함수 그 자체라고 할 수 있습니다. **가설은 알고리즘이 훈련 데이터를 통해 발견하고자 하는 입력과 출력 간의 관계를 나타내는 함수입니다.** 모델이 학습을 마쳤다는 것은, 새로운 데이터가 주어졌을 때 이 결정 경계를 기준으로 어느 클래스에 속할지 예측할 준비가 되었다는 뜻입니다.

결정 경계의 종류

결정 경계는 데이터의 분포와 사용하는 모델의 종류에 따라 그 형태가 달라집니다.

- **선형 결정 경계 (Linear Decision Boundary):** 데이터가 직선(2차원), 평면(3차원), 또는 초평면(Hyperplane, 4차원 이상)으로 분리될 수 있는 경우입니다. 이는 가장 단순하고 해석이 용이한 형태의 경계입니다. 퍼셉트론, 로지스틱 회귀, 서포트 벡터 머신(기본 형태) 등이 선형 결정 경계를 만듭니다.
- **비선형 결정 경계 (Non-linear Decision Boundary):** 데이터가 복잡하게 얽혀 있어 단순한 직선으로는 나눌 수 없는 경우입니다. 이 경우, 커널 기법을 사용하는 SVM, 결정 트리, 또는 다층 신경망과 같은 모델을 사용하여 복잡한 곡선 형태의 결정 경계를 학습해야 합니다. [스탠포드 CS221 강의 자료에 따르면](#), 복잡한 특성(feature)을 추가함으로써 원래 공간에서는 비선형이었던 결정 경계를 더 높은 차원의 특성 공간에서는 선형으로 만들 수 있습니다.

왼쪽: 두 클래스를 직선으로 나누는 선형 결정 경계(Binary Classification). 오른쪽: 세 개 이상의 클래스를 구분하는 다중 클래스 분류(Multi-class Classification)

고차원 공간에서의 결정 경계인 초평면(Hyperplane)은 n 차원 공간을 둘로 나누는 $(n-1)$ 차원의 부분 공간입니다. 예를 들어, 3차원 공간에서는 2차원 평면이 초평면이 되고, 2차원 공간에서는 1차원 직선이 초평면이 됩니다. [이 초평면은 SVM과 같은 여러 분류 알고리즘의 핵심 개념입니다.](#)

3차원 특성 공간에서 두 데이터 그룹(빨간 점과 파란 점)을 분리하는 2차원 평면, 즉 초평면(Hyperplane)의 시각적 예시

2. 최고의 경계를 찾는 여정: 손실 함수와 최적화

특성 공간에 무수히 많은 결정 경계를 그릴 수 있다면, 그중 어떤 것이 '최고'의 경계라고 할 수 있을까요? 머신러닝은 이 질문에 답하기 위해 '손실 함수'와 '최적화'라는 두 가지 강력한 수학적 도구를 사용합니다.

손실 함수(Loss Function)의 필요성

손실 함수(Loss Function), 또는 비용 함수(Cost Function)는 현재 모델(결정 경계)이 얼마나 '나쁜지'를 정량적으로 측정하는 척도입니다. 즉, 모델의 예측이 실제 정답과 얼마나 차이가 나는지를 숫자로 나타내는 함수입니다. [DataCamp의 설명에 따르면](#), 손실 함수는 모델의 성능과 정확성을 측정 가능한 방식으로 평가하며, 학습 과정의 가이드 역할을 합니다.

손실 함수의 값이 크다는 것은 모델의 오차가 크다는 의미이며, 반대로 값이 작을수록 모델이 데이터를 잘 설명하고 있다는 뜻입니다. 따라서 머신러닝 모델의 학습 목표는 이 손실 함수의 값을 가능한 한 최소화하는 것입니다. [손실 함수는 신경망이 훈련 데이터를 얼마나 잘 모델링하는지 측정합니다.](#)

대표적인 손실 함수: 힌지 손실(Hinge Loss)

다양한 손실 함수 중, 특히 서포트 벡터 머신(SVM)과 같은 최대 마진 분류기에서 주로 사용되는 것이 ****힌지 손실(Hinge Loss)****입니다. 힌지 손실의 핵심 아이디어는 단순히 예측이 맞고 틀리는 것을 넘어, 결정 경계로부터 얼마나 '확신을 갖고' 올바르게 분류했는지를 고려하는 것입니다.

힌지 손실은 다음과 같이 작동합니다 ([GeeksforGeeks, Hinge-loss & relationship with SVM](#)):

- **정확하고 확신에 찬 예측:** 데이터가 올바른 클래스로 분류되고, 결정 경계로부터 안전 마진(safety margin)보다 멀리 떨어져 있으면 손실(loss)은 0입니다. 모델은 이 데이터에 대해 페널티를 받지 않습니다.

- **정확하지만 확신이 부족한 예측:** 데이터가 올바르게 분류되었지만, 안전 마진 안쪽에 (즉, 결정 경계에 너무 가깝게) 위치한 경우, 그 거리에 비례하는 작은 손실이 발생합니다.
- **부정확한 예측:** 데이터가 잘못된 클래스로 분류된 경우, 결정 경계에서 얼마나 멀리 떨어져 있는지에 따라 선형적으로 증가하는 큰 손실이 발생합니다.

이러한 특성 때문에 힌지 손실은 모델이 단순히 데이터를 나누는 것을 넘어, 클래스 간의 간격(마진)을 최대화하도록 유도합니다. 그래프 모양이 경첩(hinge)처럼 생겼기 때문에 이런 이름이 붙었습니다. **예측값이 크게 틀릴수록 손실값은 선형으로 증가하는 특성이 있습니다.**

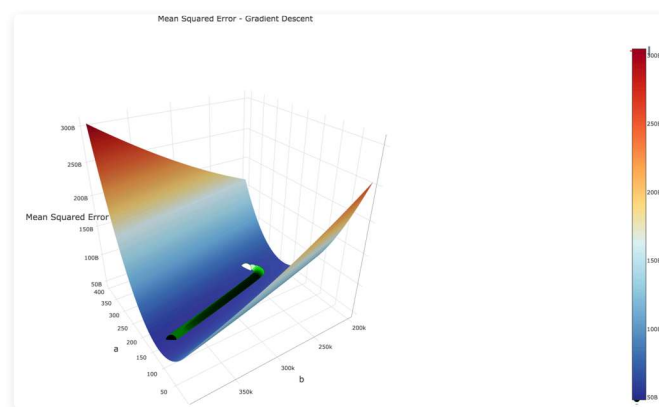
최적화(Optimization)와 경사 하강법(Gradient Descent)

최적화(Optimization)는 손실 함수를 최소화하는 모델의 최적 파라미터(예: 결정 경계를 정의하는 직선의 기울기와 절편)를 찾는 전 과정입니다. 수많은 최적화 알고리즘 중 가장 기본적이고 널리 사용되는 것이 바로 ****경사 하강법(Gradient Descent)****입니다.

경사 하강법의 원리는 '눈을 가리고 산에서 가장 낮은 지점으로 내려가는 과정'에 비유할 수 있습니다. **이 알고리즘은 함수의 기울기(gradient)의 반대 방향으로 반복적으로 이동하여 함수의 지역 최솟값을 찾는 1차 최적화 알고리즘입니다.**

1. **현재 위치 파악:** 모델의 파라미터를 무작위로 초기화합니다. 이는 산 중턱의 임의의 지점에서 시작하는 것과 같습니다.
2. **가장 가파른 방향 찾기:** 현재 위치에서 손실 함수의 ****기울기(Gradient)****를 계산합니다. 기울기는 함수 값이 가장 가파르게 증가하는 방향과 그 정도를 나타내는 벡터입니다.
3. **한 걸음 내려가기:** 기울기의 정반대 방향, 즉 손실 값이 가장 가파르게 감소하는 방향으로 ****학습률 (Learning Rate)****이라는 정해진 보폭만큼 파라미터를 업데이트(이동)합니다.
4. **반복:** 기울기가 거의 0에 가까워져 더 이상 내려갈 곳이 없을 때까지(즉, 손실 함수의 최저점에 도달할 때까지) 2~3번 과정을 반복합니다.

Google의 머신러닝 단기집중과정에서는 이 과정을 통해 모델이 수렴(converge)할 때까지, 즉 손실이 더 이상 줄어들지 않을 때까지 훈련이 계속된다고 설명합니다. 이 반복적인 과정을 통해 모델은 점진적으로 최적의 결정 경계를 찾아가게 됩니다.



손실 함수(Mean Squared Error)의 3D 표면. 경사 하강법은 임의의 지점에서 시작하여 기울기가 가장 낮은 방향을 따라 반복적으로 이동하며 함수의 최저점(가장 어두운 파란색 영역)을 찾아가합니다.

3. [핵심] 결정 경계를 만드는 알고리즘: 판별 모델 파헤치기

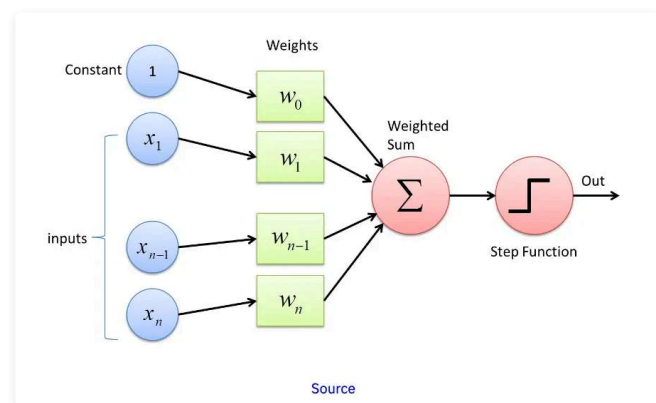
지금까지 결정 경계의 개념과 그것을 최적화하는 방법을 살펴보았습니다. 이제 실제로 결정 경계를 만들어내는 구체적인 알고리즘, 즉 '판별 모델'의 세계로 들어가 보겠습니다.

판별 모델(Discriminative Model)이란?

판별 모델(Discriminative Model)은 데이터의 생성 과정이나 분포를 학습하기보다는, 주어진 데이터의 클래스(레이블)를 구분하는 ****결정 경계**** 자체를 직접적으로 학습하는 데 초점을 맞춘 모델입니다. **수학적으로 이는 조건부 확률 분포 $P(Y|X)$ 를 모델링하는 것과 같습니다.** 즉, "입력 데이터 X 가 주어졌을 때, 이 데이터가 특정 클래스 Y 에 속할 확률은 얼마인가?"라는 질문에 답하는 방식입니다. 이 장에서 다룰 퍼셉트론, SVM, 로지스틱 회귀는 모두 대표적인 판별 모델입니다.

가장 단순한 시작: 퍼셉트론(Perceptron)

퍼셉트론은 1950년대에 프랑크 로젠블랫에 의해 개발된 초기 형태의 인공 신경망으로, 가장 단순한 선형 분류기입니다. 그 구조는 다수의 입력(features)과 각 입력에 해당하는 가중치(weights)를 받아들여, 이들의 가중합(weighted sum)을 계산하고, 특정 임계값을 넘으면 1(활성화), 넘지 않으면 0(또는 -1)을 출력하는 형태입니다.



퍼셉트론의 기본 구조: 각 입력($x_1, \dots, x_n$)은 고유한 가중치($w_1, \dots, w_n$)와 곱해진 후, 편향(w_0)과 함께 합산(Σ)되어 활성화 함수(Step Function)를 통해 최종 출력을 결정합니다.

퍼셉트론의 학습 원리는 매우 직관적입니다:

1. 초기 가중치(무작위 값)로 결정 경계를 설정하고 예측을 수행합니다.
2. 예측이 틀렸을 경우, 결정 경계가 해당 데이터를 올바르게 분류하는 방향으로 조금씩 이동하도록 가중치를 업데이트합니다.
3. 모든 훈련 데이터를 올바르게 분류할 때까지 이 과정을 반복합니다.

하지만 퍼셉트론은 치명적인 한계를 가집니다. 바로 ****선형으로 분리 가능한(linearly separable) 문제****만 해결할 수 있다는 점입니다. 데이터가 복잡하게 얽혀있는 비선형 문제, 예를 들어 유명한 XOR 문제는 단층 퍼셉트론으로 해결할 수 없습니다.

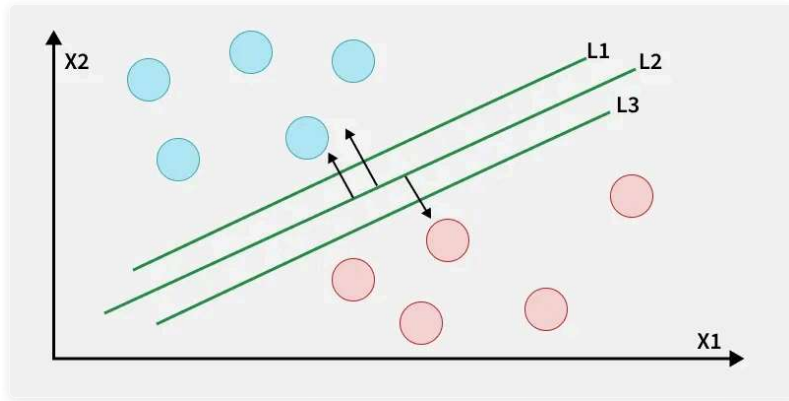
최적의 마진을 찾아서: 서포트 벡터 머신(SVM)

서포트 벡터 머신(Support Vector Machine, SVM)은 단순히 데이터를 나누는 것을 넘어, 두 클래스 사이의 간격, 즉 ****마진(Margin)****을 최대화하는 최적의 결정 경계(초평면)를 찾는 것을 목표로 합니다. **SVM의 주요 목표는 두 클래스 사이의 마진을 최대화하는 것이며, 마진이 클수록 새로운 데이터에 대한 모델의 성능이 좋아집니다.**

SVM의 핵심 개념은 다음과 같습니다:

- **초평면 (Hyperplane):** 데이터를 분류하는 결정 경계입니다.
- **서포트 벡터 (Support Vectors):** 결정 경계에 가장 가까이 위치한 데이터 포인트들로, 이들이 마진의 폭을 결정하는 데 결정적인 역할을 합니다. 이름 그대로 결정 경계를 '지지하는' 벡터들입니다.
- **마진 (Margin):** 결정 경계와 양쪽 클래스의 서포트 벡터 사이의 거리입니다. SVM은 이 마진을 최대로 만드는 초평면을 찾습니다.

또한 SVM은 ****커널 트릭(Kernel Trick)****이라는 강력한 기법을 통해 비선형 데이터 문제도 해결할 수 있습니다. 이는 저차원 특성 공간에서는 선형 분리가 불가능한 데이터를, 명시적인 계산 없이 고차원 공간으로 매핑하여 선형 분리가 가능하도록 만드는 기교입니다.



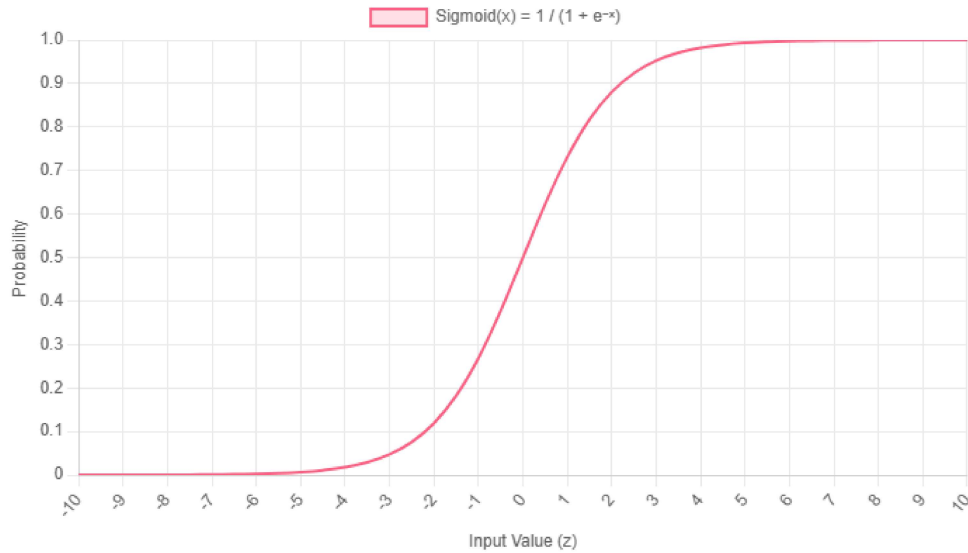
SVM은 두 클래스의 데이터를 분리하는 결정 경계(L2)를 찾되, 각 클래스에서 경계에 가장 가까운 데이터(서포트 벡터)와의 거리(마진)를 최대화합니다.

확률적 접근: 로지스틱 회귀(Logistic Regression)

이름에 '회귀'가 들어가지만, ****로지스틱 회귀****는 대표적인 분류 알고리즘입니다. 이 모델의 목표는 데이터가 특정 클래스에 속할 ****확률****을 예측하는 것입니다. 예를 들어, '이 환자가 암에 걸렸을 확률은 75%'와 같이 결과물을 확률로 제공합니다.

로지스틱 회귀의 핵심에는 ****시그모이드 함수(Sigmoid Function)****가 있습니다. 선형 모델의 예측 결과는 음의 무한대에서 양의 무한대까지 어떤 값이든 가질 수 있습니다. 시그모이드 함수는 이 값을 입력받아 항상 0과 1 사이의 값으로 변환해주는 역할을 합니다. **이 변환된 값은 특정 클래스에 속할 확률로 해석될 수 있습니다.**

분류 결정은 보통 임계값(threshold) 0.5를 기준으로 이루어집니다. 계산된 확률이 0.5보다 크면 클래스 1로, 작으면 클래스 0으로 분류합니다. 바로 이 확률이 0.5가 되는 지점들이 모여 특성 공간에서 결정 경계를 형성하게 됩니다.



[그림 10] S자 형태의 시그모이드 함수 그래프. 모든 입력 값(x 축)을 0과 1 사이의 확률 값(y 축)으로 변환하며, 입력이 0일 때 0.5의 값을 가집니다.

4. 모델 비교: 판별 모델 vs. 생성 모델

머신러닝 분류 모델은 크게 판별 모델과 생성 모델, 두 가지 접근 방식으로 나눌 수 있습니다. 이 둘은 데이터를 이해하고 문제를 해결하는 방식에 근본적인 차이가 있습니다.

두 모델의 근본적인 차이점

• 판별 모델 (Discriminative Models):

- 질문: "두 클래스의 차이점은 무엇인가?"
- 접근법: 클래스 간의 경계, 즉 결정 경계를 직접 학습합니다. 입력 X 가 주어졌을 때 출력 Y 가 나올 조건부 확률 $P(Y|X)$ 를 모델링합니다. **판별 모델은 입력 특성과 출력 레이블 간의 관계를 포착하여 새로운 데이터의 클래스를 결정합니다.**
- 비유: 영어 문장과 프랑스어 문장을 보고, 두 언어를 구별하는 '규칙'만 학습합니다. 프랑스어 문장을 새로 만들어내지는 못합니다.
- 장점: 일반적으로 분류 작업에서 더 높은 성능을 보이며 계산적으로 효율적입니다.
- 예시: 로지스틱 회귀, 서포트 벡터 머신(SVM), 결정 트리, 대부분의 신경망.

• 생성 모델 (Generative Models):

- 질문: "각 클래스의 데이터는 어떻게 생겼는가?"
- 접근법: 각 클래스 데이터의 분포 자체를 학습합니다. 입력 X 와 출력 Y 의 결합 확률 $P(X, Y)$ 를 모델링합니다. 이로부터 베이지 정리를 이용해 $P(Y|X)$ 를 유도할 수도 있습니다. **생성 모델은 개별 클래스의 분포를 모델링하는 반면, 판별 모델은 클래스 간의 경계를 학습합니다.**
- 비유: 영어의 문법, 어휘, 구조를 모두 학습하여 새로운 영어 문장을 '생성'할 수 있는 능력을 갖추는 것입니다.
- 장점: 데이터의 구조를 더 깊이 이해하므로, 이상치 탐지(outlier detection), 결측치 처리, 새로운 데이터 생성(data generation) 등에 활용될 수 있습니다.
- 예시: 나이브 베이지 분류기(Naive Bayes Classifier), 가우시안 혼합 모델(GMM), 생성적 적대 신경망(GAN).

판별 모델의 접근 방식: 특성 공간에 분포된 데이터들을 가장 잘 나눌 수 있는 하나의 최적 결정 경계(가설)를 가능한 여러 가설들(가설 공간) 속에서 찾아냅니다. 반면 생성 모델은 각 데이터 군집의 분포 자체를 모델링합니다.

언제 무엇을 사용해야 할까?

모델 선택은 해결하고자 하는 문제의 종류에 따라 달라집니다.

- 명확한 ****분류(Classification)****가 주된 목표라면, 일반적으로 더 높은 정확도를 보이는 ****판별 모델****이 선호됩니다. 예를 들어, 스팸 필터링이나 이미지 분류와 같은 작업에는 판별 모델이 더 효과적입니다.
- 데이터셋에 ****결측치가 많거나****, 훈련 데이터가 부족한 상황에서 ****이상치를 탐지****해야 하거나, 혹은 ****새로운 데이터를 생성****하는 것이 목표라면(예: 가짜 얼굴 이미지 생성), 데이터의 근본적인 분포를 학습하는 ****생성 모델****이 더 유용할 수 있습니다. [DataCamp 블로그에 따르면](#), 생성 모델은 데이터가 부족하거나 새로운 카테고리를 추가해야 하는 경우에 선호될 수 있습니다.

결론적으로, 두 모델은 상호 배타적인 관계가 아니라 상호 보완적인 관계에 있으며, 문제의 특성과 목표에 따라 적절한 모델을 선택하는 것이 중요합니다.

5. 실전! 코드로 결정 경계 시각화하기 (날씨 예측 예제)

이론적 개념을 실제로 확인하는 가장 좋은 방법은 코드로 직접 구현해보는 것입니다. 여기서는 파이썬(Python)의 대표적인 머신러닝 라이브러리인 Scikit-learn을 사용하여, 간단한 날씨 데이터를 기반으로 로지스틱 회귀 모델을 학습시키고 그 결정 경계를 시각화하는 과정을 살펴보겠습니다. 이를 통해 추상적인 개념들이 어떻게 코드와 그래프로 현실화되는지 확인할 수 있습니다.

목표 및 프로세스

이번 실습의 목표는 특정 기상 조건(예: 습도, 기온)에 따라 다음 날 비가 올지 안 올지를 예측하는 분류 모델을 만들고, 그 모델이 어떤 기준으로 '비 올'과 '비 안 올'을 판단하는지, 즉 결정 경계를 눈으로 확인하는 것입니다. [실제 날씨 예측 연구에서도 로지스틱 회귀는 기본적인 머신러닝 기법으로 활용됩니다.](#)

전체 프로세스는 다음과 같습니다.

1. **라이브러리 임포트:** 데이터 처리를 위한 `pandas`, 수치 연산을 위한 `numpy`, 시각화를 위한 `matplotlib`, 그리고 모델링을 위한 `sklearn`을 불러옵니다.
2. **데이터 준비:** 날씨 관련 데이터셋을 로드합니다. 여기서는 설명의 편의를 위해 '습도(Humidity)'와 '기온(Temperature)' 두 가지 특성만 사용하여 다음 날 강수 여부('RainTomorrow')를 예측한다고 가정합니다. 강수 여부는 '비 올(1)'과 '비 안 올(0)'으로 레이블링됩니다.
3. **모델 학습:** `sklearn.linear_model`의 `LogisticRegression` 클래스를 사용하여 준비된 데이터로 모델을 학습(fit)시킵니다.
4. **결정 경계 시각화:** 학습된 모델을 이용해 특성 공간의 각 지점이 어떤 클래스로 예측되는지 계산하고, 이를 등고선 플롯(contour plot)으로 그려 결정 경계를 시각화합니다.

날씨 데이터의 여러 변수 간 상관관계를 보여주는 히트맵. 모델 학습에 앞서 'rain_tomorrow'(내일 비)와 다른 변수들(습도, 온도 등)의 관계를 파악하는 것은 중요한 데이터 탐색 과정입니다.

결정 경계 시각화 코드 및 해설

아래는 결정 경계를 시각화하는 핵심 파이썬 코드입니다. 이 코드는 모델이 학습되었다고 가정한 후에 실행됩니다.

```
# 필요한 라이브러리 임포트
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification

# 가상 데이터 생성 (X: 특성, y: 레이블)
# 실제로는 날씨 데이터를 로드하여 사용합니다.
X, y = make_classification(n_samples=1000, n_features=2, n_redundant=0,
                           n_informative=2, random_state=1, n_clusters_per_class=1)

# 로지스틱 회귀 모델 생성 및 학습
model = LogisticRegression()
model.fit(X, y)

# 1. 특성 공간을 나눌 메쉬그리드(meshgrid) 생성
# X[:, 0]은 첫 번째 특성(예: 습도), X[:, 1]은 두 번째 특성(예: 기온)
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
# x축과 y축에 대해 0.02 간격으로 점들을 생성
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                      np.arange(y_min, y_max, 0.02))

# 2. 그리드의 모든 점에 대해 모델로 예측 수행
# xx.ravel()과 yy.ravel()은 그리드 좌표를 1차원 배열로 펼쳐줌
# np.c_는 두 배열을 열 방향으로 합쳐서 (n_samples, 2) 형태의 배열로 만듦
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
# 예측 결과를 다시 그리드 형태로 변환
Z = Z.reshape(xx.shape)

# 3. 결정 경계 및 데이터 포인트 시각화
plt.figure(figsize=(10, 7))
# contourf 함수는 예측 결과(Z)에 따라 각 영역을 다른 색으로 칠함
plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)

# scatter 함수로 실제 데이터 포인트를 겹쳐서 그림
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, edgecolors='k')

# 그래프 제목 및 축 레이블 설정
plt.xlabel('Humidity (Feature 1)')
plt.ylabel('Temperature (Feature 2)')
plt.title('Logistic Regression Decision Boundary')
plt.show()
```

이 코드를 실행하면 습도와 기온을 두 축으로 하는 2차원 평면 위에, 모델이 예측한 '비 올' 영역과 '비 안 올' 영역이 서로 다른 색으로 칠해진 그래프가 나타납니다. 두 색깔 영역 사이의 경계선이 바로 우리 모델이 학습한 ****결정 경계****입니다. 또한, 실제 데이터 포인트들이 산점도로 함께 표시되어, 모델이 데이터를 얼마나 잘 분류하고 있는지 직관적으로 확인할 수 있습니다. [이러한 결정 표면 플롯은 주어진](#)

모델이 예측 작업을 어떻게 '바라보는지' 그리고 입력 특성 공간을 어떻게 나누었는지 이해하는 강력한 진단 도구입니다.

결론 및 핵심 용어 정리

핵심 요약 및 향후 학습 방향

본문에서는 머신러닝의 근본적인 목표가 데이터가 존재하는 **특성 공간** 내에서 클래스를 구분하는 최적의 **결정 경계**를 찾는 것임을 확인했습니다. 이 과정은 수학적 함수로 표현되는 모델(가설)의 좋고 나쁨을 **손실 함수**로 정량화하고, **경사 하강법**과 같은 **최적화** 알고리즘을 통해 손실을 최소화하는 파라미터를 반복적으로 찾아나가는 여정입니다.

특히 **판별 모델**인 퍼셉트론, SVM, 로지스틱 회귀가 각각 어떤 철학(선형 분리, 마진 최대화, 확률 예측)을 바탕으로 결정 경계를 형성하는지 살펴보았습니다. 또한, 결정 경계를 직접 학습하는 판별 모델과 데이터의 분포 자체를 학습하는 **생성 모델**의 차이점도 비교했습니다.

이 글에서 다룬 기본 개념들은 더 복잡한 머신러닝 및 딥러닝 모델을 이해하는 초석이 됩니다. 향후에는 다음과 같은 주제로 학습을 확장해 나갈 수 있습니다.

- **다층 퍼셉트론(MLP)과 역전파(Backpropagation):** 다층 퍼셉트론의 한계를 극복하고 비선형 결정 경계를 학습하는 신경망의 기본 원리.
- **다양한 최적화 알고리즘:** 기본 경사 하강법의 한계를 개선한 Momentum, AdaGrad, Adam 등의 고급 최적화 기법.
- **더 복잡한 모델의 구조:** Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN) 등 특정 데이터 유형에 특화된 모델들이 어떻게 특성을 추출하고 결정 경계를 학습하는지 탐구.

핵심 용어 정리 (Glossary)

결정 경계 (Decision Boundary)

데이터 클래스를 구분하는 특성 공간상의 경계면. 모델이 예측을 내리는 기준선입니다. ([참고: 머신러닝과 분류](#))

특성 공간 (Feature Space)

데이터의 각 특성(feature)을 하나의 축으로 하는 다차원 공간. 모든 데이터는 이 공간의 한 점으로 표현됩니다. ([참고: ScienceDirect](#))

판별 모델 (Discriminative Model)

데이터의 분포보다 클래스 간의 경계를 직접 학습하는 모델. 조건부 확률 $P(Y|X)$ 를 모델링합니다. ([참고: Turing.com](#))

생성 모델 (Generative Model)

각 클래스의 데이터 분포 자체를 학습하여 새로운 데이터를 생성할 수 있는 모델. 결합 확률 $P(X,Y)$ 를 모델링합니다. ([참고: Medium](#))

손실 함수 (Loss Function)

모델의 예측 값과 실제 값의 차이, 즉 오차를 측정하는 함수. 이 함수의 값을 최소화하는 것이 학습의 목표입니다. ([참고: Wikipedia](#))

최적화 (Optimization)

손실 함수를 최소화하는 모델의 최적 파라미터를 찾는 과정입니다. ([참고: CS231n 강의 정리](#))

경사 하강법 (Gradient Descent)

손실 함수의 기울기(gradient)를 이용해 점진적으로 최적의 파라미터를 찾아가는 대표적인 최적화 알고리즘입니다. ([참고: 경사하강법 메커니즘](#))

퍼셉트론 (Perceptron)

가장 기본적인 선형 분류기이자 초기 인공 신경망 모델. 선형 분리 가능한 문제만 해결할 수 있습니다. ([참고: 퍼셉트론 개념](#))

서포트 벡터 머신 (SVM)

클래스 간의 마진(margin)을 최대화하는 결정 경계를 찾는 강력한 분류 모델입니다. ([참고: GeeksforGeeks](#))

로지스틱 회귀 (Logistic Regression)

시그모이드 함수를 사용해 데이터가 특정 클래스에 속할 확률을 예측하는 분류 모델입니다. ([참고: AWS](#))

초평면 (Hyperplane)

3차원 이상의 공간에서 정의되는 평면. n 차원 공간의 $(n-1)$ 차원 부분 공간을 의미하며, SVM 등에서 결정 경계로 사용됩니다. ([참고: Hyperplanes in ML](#))